RAG Pipeline - Step 0: Chunking Strategies

1. Definition

Chunking (or Text Splitting) is the critical pre-indexing step where large documents are broken down into smaller, manageable segments.

This is necessary because:

1. **Context Windows:** LLMs have limited input sizes.
2. **Retrieval Precision:** Retrieving a specific, relevant paragraph is far more useful to the LLM than retrieving a whole 50-page PDF.

The goal is to create chunks that are **semantically complete**—meaning a chunk should make sense on its own without needing the surrounding text.

2. Key Configuration: Chunk Overlap

Before choosing a strategy, it is critical to understand the **Overlap** parameter.

- **What is it?** A setting (e.g., `chunk_overlap=200`) that forces the splitter to repeat the last N characters/tokens of one chunk at the *beginning* of the next chunk.
- **Why use it?** It acts as a "link" between chunks. If a logical thought or sentence is cut in half by a hard split, the overlap ensures that the full thought exists intact in at least one of the chunks.
- **Typical Value:** 10% - 20% of the chunk size (e.g., for a chunk size of 1000, an overlap of 100-200 is common).

3. Heuristic Strategies (Rule-Based)

These are fast, free, and efficient. They rely on characters and separators.

A. Character / Token Splitter (Fixed Size)

- **How it works:** Simply cuts text after a fixed number of characters (e.g., every 1000 chars).
- **Pros:** Extremely fast.
- **Cons:** Often cuts sentences in half, destroying meaning ("context fragmentation").

B. Recursive Character Text Splitter (The Standard)

- **How it works:** This is the default for most RAG applications. It attempts to keep related text together by trying a list of separators in order:
    1. `\n\n` (Paragraphs) - *Try this first.*
    2. `\n` (Sentences/Lines) - *If chunk is still too big, try this.*
    3. ` ` (Words) - *Then this.*

4. `` (Characters) - *Last resort.*

- **Pros:** Balances speed with semantic coherence. It respects natural document structure like paragraphs.
- **LangChain Tool:** `RecursiveCharacterTextSplitter`.

C. Document Specific Splitters (Code/Markdown)

- **How it works:** These are "Recursive" splitters aware of specific syntax.
  - **Markdown:** Splits on headers (`#`, `##`, `###`). This ensures a Header and its content stay together.
  - **Python/JS:** Splits on class definitions (`class`), function definitions (`def`, `function`), and logical blocks.
- **Pros:** Essential for technical documentation or codebases to prevent breaking code logic.
- **LangChain Tool:** `RecursiveCharacterTextSplitter.from_language(...)` or `MarkdownHeaderTextSplitter`.

4. Semantic Strategies (Embedding-Based)

These methods use the *meaning* of the text to decide where to split, rather than just punctuation.

A. Semantic Chunking

- **How it works:**
  1. Split text into sentences.
  2. Generate an **embedding** for every sentence.
  3. Compare the cosine similarity of adjacent sentences.
  4. **Breakpoint Detection:** If the similarity between Sentence A and Sentence B drops below a certain threshold (a "breakpoint"), it assumes the topic has changed and starts a new chunk.
- **Pros:** Creates very coherent chunks based on actual topic shifts.
- **Cons:** Slower than recursive splitting because it requires embedding every sentence first.
- **LangChain Tool:** `SemanticChunker` (often uses `Percentile` or `StandardDeviation` thresholds).

5. Agentic Strategies (LLM-Based)

This is the "smartest" but most expensive category.

A. Agentic (Proposition-Based) Chunking

- **Concept:** Treats chunking as a reasoning task. A chunk shouldn't just be a "paragraph"; it should be a "complete idea."
- **How it works (The Process):**

1. **Proposition Extraction:** An LLM scans the text and breaks complex sentences into independent "atomic statements" or propositions.
    - *Input:* "LangChain, a framework for LLMs, was launched in 2022."
    - *Propositions:* ["LangChain is a framework for LLMs.", "LangChain was launched in 2022."]
2. **Agentic Grouping:** An agent processes these propositions one by one. For each new proposition, it asks: *"Does this statement belong to the current chunk's context, or does it start a new topic?"*
3. **Refinement:** If it starts a new topic, a new chunk is created.

- **Pros: Highest Accuracy.** It handles "interleaved" text perfectly (e.g., a document that switches back and forth between two topics) by logically grouping related facts regardless of where they appear.
- **Cons: Slow & Expensive.** It requires intensive LLM calls (parsing every sentence), making it unsuitable for massive datasets unless processed offline.
- **LangChain Tool:** Experimental support via `PropositionalRetriever` or custom chains using `extraction` (Source 4.3).